

Algorytm genetyczny na GPU dla N-wymiarowej kostki Rubika

Architektura, sposób działania i implementacja potoku obliczeniowego OpenGL Compute

Robert Świta

Politechnika Koszalińska

Katedra Systemów Multimedialnych i Sztucznej Inteligencji

Dokument opisuje potok GPU-GA solvera: rozdział renderowania i obliczeń, reprezentację danych, pętlę ewolucyjną, metrykę fitnessu (z karą martwego ruchu i przeważeniem końcówki), mechanizm wychodzenia z minimów lokalnych (stall/kop), jedno źródło prawdy dla oceny kostki na GPU oraz seedowanie i kluczowe decyzje implementacyjne.

1. Wprowadzenie

Solver układa N-wymiarową kostkę Rubika (rozmiar SIZE na wymiar, $SIZE^N$ kubików) za pomocą algorytmu genetycznego, w którym osobnikiem jest sekwencja ruchów. Ewaluacja populacji jest problemem wybitnie równoległym: POPULATION_COUNT osobników, każdy wykonuje GENES_COUNT ruchów na CUBIES_COUNT kubikach, całkowicie niezależnie od pozostałych. Taka charakterystyka idealnie pasuje do GPU.

Dlatego cała ewaluacja, selekcja, krzyżowanie i sortowanie zostały przeniesione na GPU jako shadery obliczeniowe OpenGL (compute shaders, GLSL 4.30). CPU pełni rolę sterownika: przygotowuje bufory, buduje seedy (sekwencje cofające orientacje kubików), wyzwala kolejne przebiegi (dispatch), odczytuje najlepszego osobnika i integruje wynik z nadrzędną pętlą rozwiązywania kostki klastrow po klastrze.

2. Architektura

Kod dzieli się na dwie rozłączne warstwy:

- **Renderowanie per-okno** (TGLContext) — kontekst OpenGL związany z konkretnym oknem (HDC/HRC), rysowanie sceny 3D. Teoretycznie może istnieć wiele okien patrzących na tę samą scenę.
- **Potok GPU-GA** (statyczna klasa Gpu) — programy shaderów, bufory SSBO/UBO oraz pętla algorytmu genetycznego (ExecuteGA). Wszystko statyczne i współdzielone — nie należy do żadnego konkretnego okna.

TGLContext.Handle tworzy kontekst GL, ustawia go jako bieżący i wywołuje Gpu.Init() — kompilacja shaderów wymaga aktywnego kontekstu. Init tworzy raz obiekty programów i shaderów oraz rezerwuje bufory, a BuildShaders osobno kompiluje i linkuje te same obiekty; dzięki temu zmiana N/SIZE tylko rekompiluje, bez tworzenia i zwalniania obiektów GL.

2.1 Pliki shaderów

Shadery to zasoby osadzone w assembly. Wspólny nagłówek Setup.glsl.c (dołączany dyrektywą #include podmienianą przez host) zawiera define'y, struktury, deklaracje buforów i funkcje pomocnicze — w tym scoreState, kanoniczną funkcję oceny stanu kostki. Programy obliczeniowe:

- Init.glsl.c — inicjalizacja populacji: pierwsze numSeeds osobników z seedów, reszta losowa,
- EvaluateMicro.glsl.c / EvaluateMacro.glsl.c — ewaluacja (dwie strategie równoległości),
- Sort.glsl.c — sortowanie bitoniczne z wbudowaną karą unikalności,
- SelCrossover.glsl.c — selekcja, krzyżowanie i makro-mutacja,
- ScoreCube.glsl.c — obliczenie baseline cube.Score na GPU (jedno źródło prawdy dla metryki).

2.2 Konfiguracja przez #define i rekompilacja

Parametry są stałymi kompilacji w Setup.glsl.c, bo określają rozmiary tablic w shaderze (np. local_cubies[CUBIES_COUNT]). Wartości N, SIZE i CUBIES_COUNT są wstrzykiwane z bieżącej kostki (SetDefineValue) przed kompilacją. Gdy użytkownik zmieni wymiar lub rozmiar, CreateBuffers wykrywa różnicę (CompiledN/CompiledSize) i wyzwala rekompilację — shader zawsze pasuje do kostki. Tu też zdefiniowane są progi zachowania: MUTATION_RATIO, SEED_RATIO, WINNERS_RATIO i STALL_LIMIT.

3. Reprezentacja danych

3.1 Osobnik i ruch

Osobnik (chromosom) to struktura Specimen. Fitness jest przechowywany jako bity float (floatBitsToUint), dzięki czemu sortowanie bitoniczne może porównywać go jako uint (dla nieujemnych wartości porządek wzorca bitowego pokrywa się z porządkiem liczb).

```
struct Specimen {
    uint Fitness;           // floatBitsToUint(fitness)
    uint MovesCount;        // dlugosc najlepszego prefiksu
    uint Moves[GENES_COUNT]; // sekwencja zakodowanych ruchow
};
// Kod ruchu: angle(2b) | plane(5b) | axis(3b) | slice(reszta)
Move getMove(uint code) {
    uint angle = code & 3u;
    uint plane = (code >> 2u) & 31u;
    uint axis  = (code >> 7u) & 7u;
    uint slice = code >> 10u;
    return Move(axis, slice, plane, angle);
}
```

```
}
```

Uwaga: żaden kod nie oznacza ruchu tożsamościowego — każdy wzorec bitów to realny obrót 90/180/270. Nie ma osobników „nie robiących”, co zagaża kodowanie i upraszcza ewaluację (patrz kara martwego ruchu, 5.4).

3.2 Spakowana macierz orientacji kubika

Orientacja pojedynczego kubika to macierz znakowanej permutacji $N \times N$ (każdy wiersz ma dokładnie jeden niezerowy element ± 1). Pakujemy ją do jednego uint: N wierszy po BITS_PER_ROW bitów, pole wiersza = $(\text{znak} \ll \text{BITS_FOR_COL}) \mid \text{kolumna}$. Tożsamość: wiersz r koduje $(\text{kolumna}=r, \text{znak}+)$, czyli wartość pola = r . Ułożony kubik ma wartość tożsamości, a jego odległość $L1$ wynosi 0.

```
#define BITS_FOR_COL (N <= 4 ? 2 : 3) // bity na indeks kolumny
#define BITS_PER_ROW (BITS_FOR_COL + 1) // + 1 bit znaku
// Dla N <= 8: N wierszy * (<=4 bity) <= 32 bity -> mieści się w uint.
```

Dla N w zakresie $[3, 8]$ cała macierz mieści się w 32 bitach, co pozwala trzymać kubik w rejestrze i obracać go wyłącznie operacjami bitowymi.

3.3 Bufory GPU (kontrakt bindingów)

Wszystkie bufory tworzy klasa `Ssbo` w stałej kolejności, więc licznik bindingów daje indeksy 0–8 dokładnie tak, jak deklaruje `Setup.gls1.c`:

Bind	Bufor	Typ	Zawartość
0	Population	SSBO	Populacja rodziców (<code>Specimen[]</code>)
1	NewPopulation	SSBO	Dzieci — wyjście krzyżowania (ping-pong)
2	Cubies	SSBO	Spakowany stan startowy (uint na kubik)
3	FreeMoves	SSBO	Dozwolone kody ruchów klastra
4	SolvedCubies	SSBO	Indeksy już ułożonych kubików
5	ActiveCubies	SSBO	Indeksy kubików aktywnego klastra
6	Planes	UBO (std140)	Pary osi płaszczyzn obrotu (<code>ivec2[]</code>)
7	SeedMoves	SSBO	Sekwencje seedów per osobnik (<code>Init/mutacja</code>)
8	ScoreBuffer	SSBO	Wynik <code>ScoreCube</code> — <code>baseline cube.Score</code>

Bufor `Cubies` jest współdzielony (tylko do odczytu): każdy wątek kopiuje stan startowy do prywatnej kopii roboczej i mutuje wyłącznie ją — współdzielony bufor jest wolny od wyścigów. `countSolved` i `countActive` przekazywane są jako uniformy (a nie przez `.length()` bufora, bo puste SSBO daje niepewne `.length()` na różnych sterownikach).

4. Sposób działania — pętla ewolucyjna

Metoda `Gpu.ExecuteGA()` realizuje jeden przebieg algorytmu genetycznego dla bieżącej kostki i zwraca najlepszego znalezionej osobnika:

```
CreateBuffers(kostka, freeMoves) // upload + seedy + ew. rekompilacja
UseProgram(Init); Dispatch // Population[0]: numSeeds seedów + reszta losowa
for (gen = 0; gen < GENERATIONS_COUNT; ++gen) {
    UseProgram(evalProgram); Dispatch // 1. fitness każdego osobnika (z kara no-op)
    UseProgram(Sort); Dispatch* // 2. sortowanie bitoniczne + kara unikalności
    read Population[0] // 3. najlepszy na indeksie 0
    if (fitness < best) { best = ...; // zapamiętaj rekord
        if (best < Score) return best; } // prog akceptacji (rozwiązanie)
    UseProgram(SelCrossover); Dispatch // 4. selekcja + krzyżowanie + makro-mutacja
    parentSlot ^= 1; // 5. ping-pong buforów
}
return best; // brak poprawy: najmniej-zły REALNY ruch
```

Bufory populacji działają w trybie ping-pong: rodzice są zawsze pod bindingiem 0, dzieci pod bindingiem 1, a po każdej generacji role buforów się zamieniają. Kluczowe: dzięki karze martwego ruchu (5.4) zwrócony **best** jest zawsze **realnym** ruchem zmieniającym klastery — nawet gdy nic nie poprawia. To warunek działania mechanizmu stall/kop (7.2).

4.1 Wymiary dispatchu: micro vs macro

Ewaluacja ma dwie strategię równoległości, wybierane na podstawie liczby kubików:

- **Micro** (≤ 64 kubiki): 1 wątek = 1 osobnik. Kostka trzymana w prywatnej tablicy w rejestrach; wątek sekwencyjnie obraca kubiki i sumuje błęd. Dispatch: $\text{ceil}(\text{POP}/64)$ bloków po 64.
- **Macro** (> 64 kubiki): 1 blok (256 wątków) = 1 osobnik. Kostka w pamięci współdzielonej bloku; wątki równolegle obracają kubiki, a błąd liczony jest logarytmiczną redukcją równoległą. Dispatch: POP bloków.

Init i SelCrossover to zawsze 1 wątek na osobnika (bloki po 64). ScoreCube to 1 wątek (liczy baseline raz na klaster).

5. Ewaluacja i metryka fitnessu

5.1 Symulacja ruchu na spakowanej macierzy

TurnSingleCubie sprawdza, czy dany kubik leży w warstwie obracanej przez ruch, i jeśli tak — obraca jego macierz. Przynależność do warstwy wyznacza `getStartCoordinate` (współrzędna domowa kubika wzdłuż osi ruchu) skorygowana o znak. Konwencja osi jest zgodna z CPU. Sam obrót to `RotateMatInt` na dwóch wierszach płaszczyzny obrotu (zamiana/negacja pól — jedna formuła na każdy kąt $90^\circ/180^\circ/270^\circ$).

5.2 Odległość L1 i miara ruchów-do-ulożenia

Miarą błędu kubika jest **cubieState** — nie sama odległość L1, lecz liczba aktywnych osi (wierszy różnych od tożsamości) jako człon dominujący, dopiero potem L1 jako rozstrzygający remisy:

```
uint cubieL1(uint M) {
    // Manhattan od tozsamosci; 0 <=> ułożony
    uint dist = 0u;
    for (uint r = 0u; r < uint(N); r++) {
        uint field = (M >> (r*BITS_PER_ROW)) & ((1u<<BITS_PER_ROW)-1u);
        dist += (field > r) ? (field - r) : (r - field);
    }
    return dist;
}

uint activeAxes(uint M) {
    // liczba wierszy != tozsamosc
    uint m = 0u;
    for (uint r = 0u; r < uint(N); r++)
        if (((M >> (r*BITS_PER_ROW)) & ((1u<<BITS_PER_ROW)-1u)) != r) m++;
    return m;
}

uint cubieState(uint M) {
    // ruchy-do-ulozenia (dominant) + L1 (tiebreak)
    return activeAxes(M) * (uint(MAX_CUBIE_L1) + 1u) + cubieL1(M);
}
```

Kubik o m aktywnych osiach wymaga do ułożenia około $m-1$ ruchów (drzewo spinające obrotów Givensa nad jego aktywnymi osiami), więc m jest bezpośrednią miarą liczby ruchów. Ważenie go ponad cały zakres L1 sprawia, że szukanie najpierw minimalizuje **liczbę** ruchów, a dopiero w jej ramach — surową odległość orientacji. To daje ściśle monotoniczny gradient w przestrzeni ruchów, którego samo L1 nie zapewnia.

5.3 Składowe fitnessu i przeważanie końcówki

Fitness stanu (po danym prefiksie ruchów) łączy dwie części: **błąd aktywnego klastra** — suma po kubikach klastra $(\text{maxClusterState} + \text{cubieState})/\text{max_fA}$, gdzie $\text{maxClusterState} = \text{MAX_CUBIE_STATE} \cdot \text{countActive}$, $\text{max_fA} = \text{maxClusterState} \cdot (\text{countActive} + 1)$ — oraz **naruszenia ułożonych**: liczba kubików ze zbioru `SolvedCubies`, które przestały być ułożone (każdy dodaje całkowite 1). Ponieważ `MAX_CUBIE_STATE` przewyższa osiągalne maksimum, człon aktywny pozostaje ściśle < 1 , a każde naruszenie ułożonego kubika (wartość całkowita) zawsze nad nim dominuje — chroni to już ułożone klastry.

Przeważanie końcówki (anty-mono-twist). Zmniejszanie liczby nieułożonych kubików jest dobrym drogowskazem na początku, ale mylącym pod koniec: konfiguracja z bardzo mało nieułożonymi nie jest łatwiejsza, a w 4D i wyżej pojedynczy kubik może pozostać skręcony w izolacji (mono-twist), z czego bardzo trudno uciec. Gdy liczba nieułożonych aktywnych kubików spadnie do N lub mniej, błąd aktywny mnożymy przez $N/\text{scrambled}$ — człon liczności staje się stały, a szukanie przestaje gonić za mniejszą liczbą kubików i minimalizuje wzmocnioną magnitudę orientacji. To przełączenie jest niezbędne dla $N > 3$.

5.4 Kara martwego ruchu (no-op)

W minimum lokalnym fitness nagradza „nic-nie-rób” (ocena = `cube.Score` to najmniejsza osiągalna), więc populacja zbiega do osobników, których wypadkowy ruch jest tożsamością — best „nic nie robi”. Aby temu zapobiec, ewaluator **karze prefiks, który nie zmienił aktywnego klastra**. Detekcja jest niemal darmowa: porównanie spakowanej orientacji każdego aktywnego kubika ze stanem startowym (buforem `Cubies`).

```
bool changed = false;
for (uint i = 0; i < countActive; i++) {
```

```

uint cur = local_cubies[ActiveCubies[i]];
if (cur != Cubies[ActiveCubies[i]]) changed = true; // realnie ruszył klaster?
float e = GetActiveCubieError(cur, maxClusterState, max_fA);
local_fA_sum += e; if (e != 0.0) scrambled++;
}
float current = float(local_solved_errors) + local_fA_sum;
if (!changed) current += 1e6; // no-op na klastrze nie jest rozwiązaniem

```

To poprawne i bezpieczne: prefiks pozostawiający klaster nietknięty ma $\text{fitness} \geq \text{cube.Score}$ (nigdy nie poprawia), więc kara nic wartościowego nie odrzuca. W efekcie minimum ewaluatora przeskakuje z martwego ruchu na **najmniej-zły realny ruch zmieniający klaster**. Przewaga nad porównywaniem samego fitnessu jest istotna: na plateau (fitness płaski, ale układ się zmienia) porównanie fitnessu fałszywie widziałoby „brak zmiany”, a porównanie orientacji widzi realny ruch i pozwala kopnąć dokładnie tam, gdzie jesteśmy zakleszczeni. W macro-ewaluatorze flaga `changed` jest redukowana przez blok (OR).

6. Selekcja, krzyżowanie, sortowanie, seedowanie

6.1 Sortowanie bitoniczne z karą unikalności

Populacja jest sortowana rosnąco względem fitnessu siecią bitoniczną (`POPULATION_COUNT` jest potęgą 2). Zbiegająca populacja szybko zapełnia się klonami o równym fitnessie, co zwęża pulę rodziców i prowadzi do przedwczesnej zbieżności. Zamiast osobnego przebiegu deduplikacji, **kara unikalności wpięta jest wprost w porównanie**: gdy porównanie znajduje dwa osobniki o równym fitnessie, jeden dostaje karę równą maksymalnemu fitnessowi (`CUBIES_COUNT`). Osobniki o różnym fitnessie wypływają na górę, duplikaty toną poza pulę zwycięzców. Ponieważ kara tylko zwiększa fitness, globalne minimum nigdy nie jest karane i indeks 0 pozostaje prawdziwym najlepszym; kara jest przejściowa (nadpisywana przy następnej ewaluacji). Ta jedna zmiana była decydująca w praktyce.

6.2 Selekcja i krzyżowanie (SelCrossover)

Każdy wątek tworzy jedno dziecko. **Selekcja rankingowa**: dwaj różni rodzice losowani z górnych `WINNERS_RATIO%` posortowanej populacji. **Krzyżowanie odwrotno-komplementarne** buduje komutatory — grupowo-teoretyczne narzędzie do przestawiania kilku elementów bez ruszania reszty: pierwsza część genomu to split-mix genów obojga rodziców, a druga połowa to odwrócona sekwencja z zanegowanymi kątami (`getRevCode`). Wynik ma strukturę sprzężenia $A \cdot B \cdot A^{-1}$, z której populacja składa komutatory 3-cyklu i 2-orientacji domykające klastry.

6.3 Makro-mutacja

Operator mutacji to **nie** losowa zmiana genu (ta jedynie degraduje osobniki i została usunięta), lecz makro-mutacja: nadpisanie wiodących genów dziecka świeżą, pełną sekwencją układającą jeden kubik aktywnego klastra, a następnie sprzężenie jej fragmentu. Stosowana do frakcji `MUTATION_RATIO%` dzieci co generację, nieustannie wstrzykuje do populacji „solwery kubików”, które krzyżowanie rekombinuje w rozwiązania wielokubikowe. Bez tego wstrzykiwania materiał seedowy z inicjalizacji rozmywa się w ciągu kilku generacji.

6.4 Seedowanie: generator ruchów cofających orientację

Zarówno inicjalizacja, jak i makro-mutacja czerpią z podaży sekwencji układających pojedynczy kubik. Generuje je zachłanne najkrótsze-rozwiązanie losowego aktywnego kubika (`GetSolveSeq`): w każdym kroku wybieramy losowo ruch (płaszczyzna + kąt 90/180/270), który obniża `RotationCount` kubika o jeden, zapisujemy go i aplikujemy — aż orientacja stanie się tożsamością. Ponieważ `RotationCount` jest odległością grafową do stanu ułożonego, ruch-postęp zawsze istnieje na jakiejś najkrótszej ścieżce; procedura nigdy się nie zaczyna i nie wymaga walidacji. Każdy ruch realizowany jest z losową, poprawną osią kolateralną (dowolną spozza płaszczyzny obrotu) — oś nie zmienia trajektorii celu, ale różnicuje, które inne kubiki są poruszane przy okazji.

Host (`BuildSeedMoves`) losuje różne dekompozycje aż do nasycenia puli: przerywa, gdy przez pewien czas nie pojawia się nowe drzewo (dla $N=3$ różne drzewa to często ≤ 3 , więc pętla kończy się po kilkudziesięciu losowaniach zamiast kręcić w kółko). Inicjuje pierwsze `numSeeds` osobników z tak zebranych sekwencji (`SEED_RATIO%` populacji jako cel górny), reszta startuje losowo — realne ciśnienie seedujące daje jednak makro-mutacja, nie sam `Init`.

7. Integracja z pętlą klastrów (host)

Kostka rozwiązywana jest klastery po klastre. NextCluster wyznacza kolejny aktywny klastery (kubiki o najmniejszym indeksie klastra, które nie są jeszcze ułożone). Ruchy najlepszego osobnika są aplikowane na CPU (Turn), a gdy Score spadnie do 0, klastery jest ułożony i przechodzimy do następnego.

7.1 Jedno źródło prawdy dla oceny (ScoreCube)

Baseline RubikCube.Score — próg przerywania GA (gdy `best.Fitness < Score`) i akceptacji ciągu ruchów — liczony jest **na GPU** przez kernel ScoreCube, tą samą funkcją scoreState, którą realizują ewaluatory. Wcześniejsze lustrzane obliczenie po stronie CPU (EvaluateGpu + pomocnicy) zostało usunięte: przy każdej zmianie metryki trzeba było synchronizować shader i host, a rozjazd cichcem psuł warunek `best.Fitness < Score`. Teraz metryka żyje w jednym miejscu na GPU; host tylko wgrywa stan kostki, wywołuje dispatch i odczytuje jednego floata z ScoreBuffer.

7.2 Stall i „kop” — wychodzenie z minimów lokalnych

Zwykła reguła „zastosuj besta tylko gdy poprawia” to czysty zjazd zachłanny, który utyka w minimach lokalnych (stąd dawny ciężki ogon czasów). Gdy przez `STALL_LIMIT` kolejnych przebiegów GA nic się nie poprawia, host **stosuje besta mimo braku poprawy** — a ponieważ kara no-op gwarantuje, że best jest realnym ruchem, jest to prawdziwe szturchnięcie wyprowadzające z kotliny (basin-hopping). Włączenie/wyłączenie to jeden wbudowany warunek: `if (best.Fitness < Score || Stall >= STALL_LIMIT)`. Co istotne, kop odpala się przy **każdym** minimum lokalnym, nie tylko w globalnej końcówce — a każdy klastery ma własną mini-kończówkę (ostatnie sprzężone kubiki), więc ten sam mechanizm domyka klastry na każdym etapie, co czyni kostki wieloklastrowe (np. 3^4) rozwiązywalnymi.

7.3 Jedna reprezentacja stanu i cykl życia buforów

Stan kostki na CPU trzyma wyłącznie Transform (macierz orientacji + pozycja per kubik). Wcześniejsza równoległa reprezentacja SOA została usunięta jako zbędne dublowanie. PackCubies pakuje macierz do formatu shadera wprost z Transform.M (obroty 90° dają dokładne $0/\pm$ skala). Bufory tworzone są raz i reużywane: Ssbo.Update wywołuje glBufferData, które realokuje magazyn przy zmianie rozmiaru, zachowując to samo id i binding — eliminuje to churn buforów, typowe źródło niestabilności sterownika.

8. Wyniki

Solver dochodzi do trudnych konfiguracji w minutach, bez żadnej bazy zapamiętanych makro-ruchów. Trzy elementy okazały się decydujące i wzajemnie wzmacniające: metryka ruchów-do-ułożenia z przeważaniem końcówki (rozszerza metodę na $N > 3$), kara unikalności wpięta w sort (utrzymuje różnorodność), oraz para **kara no-op + kop** (wyprowadza z minimów lokalnych na każdym etapie).

Wpływ pary no-op/kop na kostkę 2^4 (jeden sprzężony klastery narożny, 10 ułożeń): względem wersji sprzed tej zmiany średnia liczba przebiegów GA spadła z ~ 27300 do ~ 3760 ($\sim 7\times$), a ogon z ~ 136000 do ~ 15900 ($\sim 8,6\times$); mediana z ~ 6500 do ~ 1400 . Kostka $6\times 6\times 6$ układa się w około 10 minut. Po raz pierwszy ułożona została też pełna 4-wymiarowa kostka 3^4 (81 kubików, wiele typów klastrów) — w około 2 minutach i 12055 przebiegach GA.

9. Uwagi implementacyjne i pułapki

- **Jedno źródło prawdy:** metrykę oceny liczy wyłącznie GPU (scoreState, używane przez ScoreCube i — w formie inline — przez oba ewaluatory). Host nie ma już lustrzanej kopii, więc nie ma dryfu CPU $\leftrightarrow$ GPU.
- **Detekcja no-op po orientacji, nie po fitnessie:** na plateau fitness jest płaski, więc tylko porównanie spakowanej macierzy poprawnie odróżnia ruch boczny (zmieniający układ) od martwego.
- **Reinterpretacja fitnessu:** GPU zapisuje floatBitsToUint(fitness), host odczytuje bitowo. Sortowanie bitoniczne działa na wzorcu bitowym (poprawne dla fitnessu ≥ 0).
- **Liczności klastra jako uniformy:** countSolved/countActive z hosta, nie z .length() bufora — pierwszy klastery ma pusty SolvedCubies, a .length() na SSBO rozmiaru 0 jest niepewne między sterownikami.
- **Wyrównanie std140:** tablica ivec2 Planes[] w UBO ma każdy element wyrównany do 16 bajtów (vec4), więc host uploaduje po 4 inty na element.
- **Obejście błędu sterownika:** w getStartCoordinate ogólny wzór musi leżeć w bloku else. Wariant z wczesnym return po `if (SIZE==2)` był miscompilowany (inline do gorącej pętli), co dawało Access Violation zależny od danych.

Podsumowanie: potok GPU-GA oddziela współdzielone obliczenia (klasa Gpu) od renderowania per-okno. Spakowana macierz w jednym uint pozwala symulować ruchy w rejestrach; metryka ruchów-do-ułożenia z przeważaniem końcówki daje gładki, monotoniczny krajobraz; kara martwego ruchu plus kop wyprowadzają z minimów lokalnych; a jedno źródło oceny na GPU znosi dryf metryki host-device.